

PROGRAM :

```
graph = {  
    '5': ['3', '7'],  
    '3': ['2', '4'],  
    '7': ['8'],  
    '2': [],  
    '4': ['8'],  
    '8': [],  
}  
  
visited = []  
queue = []  
  
def bfs(visited, graph, node):  
    visited.append(node)  
    queue.append(node)  
    while queue:  
        m = queue.pop(0)  
        print(m, end="")  
        for neighbour in graph[m]:  
            if neighbour not in visited:  
                visited.append(neighbour)  
                queue.append(neighbour)  
  
# Driver Code  
  
print("Following is the Breadth-First Search")  
  
bfs(visited, graph, '5')
```

OUTPUT :

Following is the Breadth-First Search

537248

PROGRAM :

```
def recursive_dfs(graph, source, path=None):

    if path is None:

        path = []

    if source not in path:

        path.append(source)

    if source not in graph:

        return path

    for neighbour in graph[source]:

        if neighbour not in path:

            path = recursive_dfs(graph, neighbour, path)

    return path

# Driver Code

graph = {"A": ["B", "C", "D"],

        "B": ["E"],

        "C": ["F","G"],

        "D": ["H"],

        "E": [],

        "F": ["J"],

        "G": [],

        "H": [],

        "J": []}

# Example usage:

path = recursive_dfs(graph,"A")

print("Following is the Depth-First Search")

print("".join(path))
```

OUTPUT :

Following is the Depth-First Search

ABECFJGDH

PROGRAM :

```
from collections import deque

class Graph:

    def __init__(self, adjaency_list):

        self.adjaency_list = adjaency_list

    def get_neighbors(self, v):

        return self.adjaency_list[v]

    def h(self, n):

        H = {'A': 1, 'B': 1, 'C': 1, 'D': 1, 'E': 1, 'G': 1}

        return H[n]

    def a_star_algorithm(self, start_node, stop_node):

        open_list = set([start_node])

        closed_list = set([])

        g = {}

        g[start_node] = 0

        parents = {}

        parents[start_node] = start_node

        while len(open_list)>0:

            n = None

            for v in open_list:

                if n == None or g[v] + self.h(v) < g[n] + self.h(n):

                    n = v

            if n == None:

                print('Path does not exist!')

                return None

            if n == stop_node:

                reconst_path = []

                while parents[n] != n:

                    reconst_path.append(

n = parents[n]

                reconst_path.append(start_node)
```

```

reconst_path.reverse()

print('Path found: {}'.format(reconst_path))

return reconst_path

open_list.remove(n)

closed_list.add(n)

for (m, weight) in self.get_neighbors(n):
    if m not in open_list and m not in closed_list:
        open_list.add(m)
        parents[m] = n
        g[m] = g[n] + weight
    else:
        if g[m] > g[n] + weight:
            g[m] = g[n] + weight
            parents[m] = n
        if m in closed_list:
            closed_list.remove(m)
            open_list.add(m)

print('Path does not exist!')

return None

adjacency_list = {'A': [('B', 1), ('C', 1)],
                  'B': [('D', 1), ('G', 1)],
                  'C': [('D', 1), ('G', 1)],
                  'D': [('E', 1)],
                  'G': [('E', 1)],
                  'E': []}

graph1 = Graph(adjacency_list)

graph1.a_star_algorithm('A','E')

```

OUTPUT :

Path found: ['A','C','D','E']

['A','C','D','E']

PROGRAM :

```
nodes = {
    'A': [['B', 6], ['F', 3]],
    'B': [['A', 6], ['C', 3], ['D', 2]],
    'C': [['B', 3], ['D', 1], ['E', 5]],
    'D': [['B', 2], ['C', 1], ['E', 8]],
    'E': [['C', 5], ['D', 8], ['I', 5], ['J', 5]],
    'F': [['A', 3], ['G', 1], ['H', 7]],
    'G': [['F', 1], ['I', 3]],
    'H': [['F', 7], ['I', 2]],
    'I': [['G', 3], ['H', 2], ['E', 5], ['J', 3]],
    'J': [['E', 5], ['I', 3]]
}

h = {
    'A': 10, 'B': 6, 'C': 5, 'D': 7,
    'E': 3, 'F': 6, 'G': 5,
    'H': 3, 'I': 1, 'J': 0
}

def astar(start, goal):
    open_set = {start}
    closed_set = set()
    g = {start: 0}
    parent = {start: None}
    while open_set:
        current = min(open_set, key=lambda x: g[x] + h[x])
        if current == goal:
            path = []
            while current:
                path.append(current)
                current = parent[current]
            path = path[::-1]
```



```
# f-cost = g(goal) + h(goal)

f_cost = g[goal] + h[goal]

print("Path found:", path)

print("Total estimated cost (f):", f_cost)

return path

open_set.remove(current)

closed_set.add(current)

for neighbor, cost in nodes[current]:

    if neighbor in closed_set:

        continue

    tentative_g = g[current] + cost

    if neighbor not in open_set or tentative_g < g.get(neighbor, float('inf')):

        parent[neighbor] = current

        g[neighbor] = tentative_g

        open_set.add(neighbor)

print("Path does not exist!")

return None

astar('A','J')
```

OUTPUT :

Path found: ['A','F','G','I','J']

Total estimated cost (f): 10

['A','F','G','I','J']

PROGRAM :

Load the iris dataset

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

Store the feature matrix (X) and response vector (y)

```
X = iris.data
```

```
y = iris.target
```

Splitting X and y into training and testing sets

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.4, random_state = 1)
```

Training the model on training set

```
from sklearn.naive_bayes import GaussianNB
```

```
gnb = GaussianNB()
```

```
gnb.fit(X_train , y_train)
```

Making predictions on the testing set

```
y_pred = gnb.predict(X_test)
```

Comparing actual response values (y_test) with predicted response values (y_pred)

```
from sklearn import metrics
```

```
print("Gaussian Naive Bayes Model Accuracy(in %): ", metrics.accuracy_score(y_test, y_pred)*100)
```

OUTPUT :

Gaussian Naive Bayes Model Accuracy(in %): 95.0

PROGRAM :

```
from pgmpy.models import DiscreteBayesianNetwork

from pgmpy.factors.discrete import TabularCPD

from pgmpy.inference import VariableElimination

# Create Bayesian Network

model = DiscreteBayesianNetwork([

    ('Burglary','Alarm'),

    ('Earthquake','Alarm'),

    ('Alarm','JohnCalls'),

    ('Alarm','MaryCalls')

])

# CPDs

cpd_burglary = TabularCPD(

    variable='Burglary', variable_card=2,

    values=[[0.999], [0.001]]

)

cpd_earthquake = TabularCPD(

    variable='Earthquake', variable_card=2,

    values=[[0.998], [0.002]]

)

cpd_alarm = TabularCPD(

    variable='Alarm', variable_card=2,

    values=[

        [0.95, 0.94, 0.29, 0.001],

        [0.05, 0.06, 0.71, 0.999]

    ],

    evidence=['Burglary','Earthquake'],

    evidence_card=[2, 2]

)

cpd_john = TabularCPD(

    variable='JohnCalls', variable_card=2,
```

```

values=[
    [0.90, 0.05],
    [0.10, 0.95]
],
evidence=['Alarm'],
evidence_card=[2]
)

cpd_mary = TabularCPD(
    variable='MaryCalls', variable_card=2,
    values=[
        [0.70, 0.01],
        [0.30, 0.99]
    ],
    evidence=['Alarm'],
    evidence_card=[2]
)

# Add CPDs
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)

# Check model
model.check_model()

# ---- PRINTING (EXACT FORMAT YOU ASKED) ----
print('Probability Distribution, P(Burglary)')
print(cpd_burglary)

print('Probability Distribution, P(Earthquake)')
print(cpd_earthquake)

```

```
print('Joint Probability Distribution, P(Alarm | Burglary, Earthquake)')

print(cpd_alarm)

print('Joint Probability Distribution, P(JohnCalls | Alarm)')

print(cpd_john)

print('Joint Probability Distribution, P(MaryCalls | Alarm)')

print(cpd_mary)

# Inference (optional)

infer = VariableElimination(model)

posterior1 = infer.query(
    variables=['Burglary'],
    evidence={'JohnCalls': 1, 'MaryCalls': 1}
)

posterior2 = infer.query(
    variables=['Alarm'],
    evidence={'Burglary': 1, 'Earthquake': 1}
)

print('\nPosterior Probability, P(Burglary | JohnCalls, MaryCalls)')

print(posterior1)

print('\nPosterior Probability for Alarm, P(Burglary | Earthquake)')

print(posterior2)
```

OUTPUT:

Probability Distribution, $P(\text{Burglary})$

Burglary(0)	0.999
Burglary(1)	0.001

Probability Distribution, $P(\text{Earthquake})$

Earthquake(0)	0.998
Earthquake(1)	0.002

Joint Probability Distribution, $P(\text{Alarm} \mid \text{Burglary}, \text{Earthquake})$

Burglary	Burglary(0)	Burglary(0)	Burglary(1)	Burglary(1)
Earthquake	Earthquake(0)	Earthquake(1)	Earthquake(0)	Earthquake(1)
Alarm(0)	0.95	0.94	0.29	0.001
Alarm(1)	0.05	0.06	0.71	0.999

Joint Probability Distribution, $P(\text{JohnCalls} \mid \text{Alarm})$

Alarm	Alarm(0)	Alarm(1)
JohnCalls(0)	0.9	0.05
JohnCalls(1)	0.1	0.95

Joint Probability Distribution, $P(\text{MaryCalls} \mid \text{Alarm})$

Alarm	Alarm(0)	Alarm(1)
MaryCalls(0)	0.7	0.01
MaryCalls(1)	0.3	0.99

Posterior Probability, $P(\text{Burglary} \mid \text{JohnCalls}, \text{MaryCalls})$

Burglary	phi(Burglary)
Burglary(0)	0.9911
Burglary(1)	0.0089

Posterior Probability for Alarm, $P(\text{Burglary} \mid \text{Earthquake})$

Alarm	phi(Alarm)
Alarm(0)	0.0010
Alarm(1)	0.9990

PROGRAM :

```
import matplotlib.pyplot as plt

from scipy import stats

# Data

x = [5, 7, 8, 7, 2, 17, 2, 9, 4, 11, 12, 9, 6]

y = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]

# Linear regression

slope, intercept, r, p, std_err = stats.linregress(x, y)

# Regression function

def myfunc(x):

    return slope * x + intercept

# Predicted values

mymodel = list(map(myfunc, x))

# Print results

print("Slope:", slope)

print("Intercept:", intercept)

print("Correlation coefficient (r):", r)

print("P-value:", p)

print("Standard Error:", std_err)

# Plot

plt.scatter(x, y)

plt.plot(x, mymodel)

plt.xlabel("X values")

plt.ylabel("Y values")

plt.title("Simple Linear Regression")

plt.show()
```

OUTPUT :

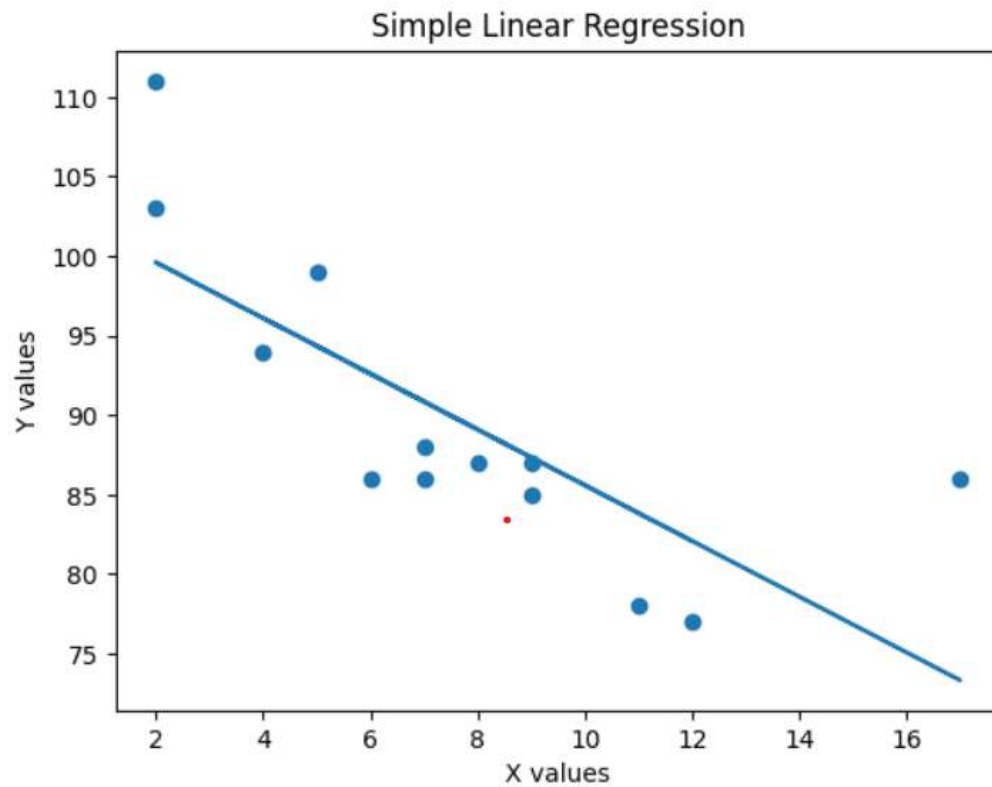
Slope: -1.7512877115526118

Intercept: 103.10596026490066

Correlation coefficient (r): -0.758591524376155

P-value: 0.0026468739224561064

Standard Error: 0.453536157607742



PROGRAM :

```
from sklearn.datasets import load_iris

from sklearn import tree

import graphviz

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

# Target names

targets = iris.target_names

# Create Decision Tree classifier

clf = tree.DecisionTreeClassifier()

# Train the model

clf = clf.fit(X, y)

# Prediction for new data

x_pred = [6.7, 3.0, 5.2, 2.3]

y_pred = clf.predict([x_pred])

print("Prediction is:", targets[y_pred[0]])

# Visualize Decision Tree

dot_data = tree.export_graphviz(

    clf,

    out_file=None,

    feature_names=iris.feature_names,

    class_names=iris.target_names,

    filled=True,

    rounded=True,

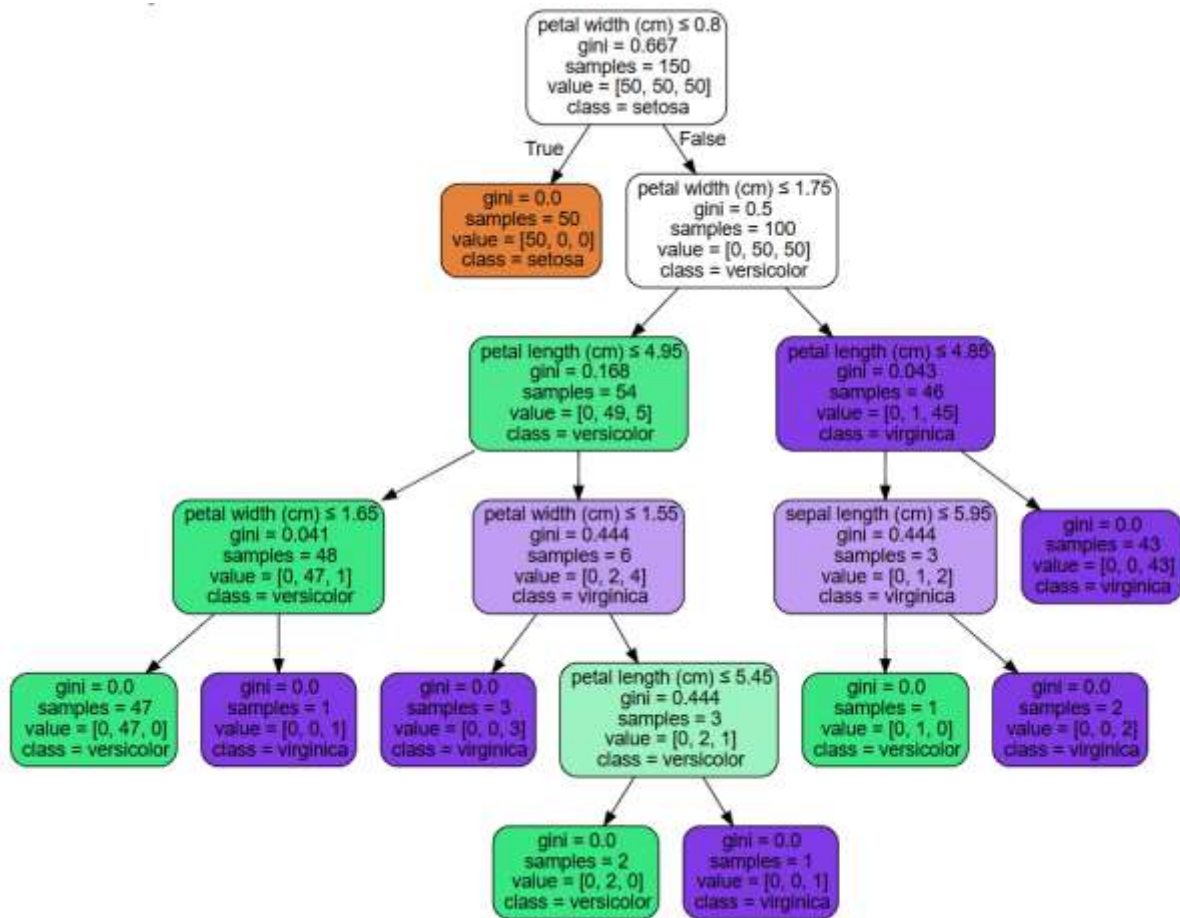
    special_characters=True

)

graph = graphviz.Source(dot_data)
```

OUTPUT :

Prediction is: virginica



PROGRAM :

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.datasets import load_iris

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

targets = iris.target_names

# Create Random Forest classifier

clf = RandomForestClassifier(random_state=100)

# Train the model

clf.fit(X, y)

# Prediction for new sample

y_pred = clf.predict([x_pred])

print("Prediction is:", targets[y_pred[0]])
```

OUTPUT :

Prediction is: virginica

PROGRAM :

```
from sklearn.datasets import load_iris

from sklearn.svm import SVC

# Load dataset

iris = load_iris()

X_train = iris.data

y_train = iris.target

targets = iris.target_names

print("Target classes:", targets)

# Create SVM classifier

cls = SVC(kernel='linear')

# Train the model

cls.fit(X_train, y_train)

# Prediction

x_pred = [5.1, 3.2, 1.5, 0.5]

y_pred = cls.predict([x_pred])

print("Prediction is:", targets[y_pred[0]])
```

OUTPUT :

Target classes: ['setosa"versicolor"virginica']

Prediction is: setosa

PROGRAM :

```
from sklearn.datasets import load_iris

from sklearn.svm import SVC

from sklearn.ensemble import RandomForestClassifier, VotingClassifier

from sklearn import tree

# Load dataset

iris = load_iris()

X_train = iris.data

y_train = iris.target

targets = iris.target_names

print("Target classes:", targets)

# Base models

m1 = tree.DecisionTreeClassifier()

m2 = RandomForestClassifier(random_state=100)

m3 = SVC(kernel='linear')

# Voting Classifier (Ensemble)

final_model = VotingClassifier(

    estimators=[

        ('dt', m1),

        ('rf', m2),

        ('svc', m3)

    ],

    voting='hard')

# Train ensemble model

final_model.fit(X_train, y_train)

# Prediction

x_pred = [6.7, 3.0, 5.2, 2.3]

y_pred = final_model.predict([x_pred])

print("Prediction is:", targets[y_pred[0]])
```

OUTPUT :

Target classes: ['setosa"versicolor"virginica']

Prediction is: virginica

PROGRAM :

```
from sklearn.datasets import load_iris

from sklearn.neighbors import KNeighborsClassifier

# Load dataset

iris = load_iris()

X_train = iris.data

y_train = iris.target

targets = iris.target_names

print("Target classes:", targets)

# Create k-NN classifier

cls = KNeighborsClassifier(n_neighbors=5)

# Train model

cls.fit(X_train, y_train)

# Prediction

x_pred = [6.7, 3.0, 5.2, 2.3]

y_pred = cls.predict([x_pred])

print("The Prediction is:")

print(targets[y_pred[0]])
```

OUTPUT :

Target classes: ['setosa' 'versicolor' 'virginica']

The Prediction is:

Virginica

PROGRAM :

```
import numpy as np

import time

# Nodes

graphNodes = ["a","b","c","d","e","f","g","h"]

# Node indices

graphNodeIndices = {node: idx for idx, node in enumerate(graphNodes)}

# Number of states per node

graphNodeNumStates = {

    "a": 3,"b": 4,"c": 5,"d": 4,

    "e": 3,"f": 4,"g": 5,"h": 4

}

# Parents

nodeParents = {

    "a": [],

    "b": [],

    "c": ["a"],

    "d": ["a","b"],

    "e": ["a","c"],

    "f": ["b","d"],

    "g": ["e"],

    "h": ["f"]

}

# Tensor order (node + parents)

tensorNodeOrder = {}

for node in graphNodes:

    tensorNodeOrder[node] = [node] + nodeParents[node]

# Random CPT generator

def randomTensorGenerator(shape):

    return np.random.uniform(0.0, 1.0, shape)

# Normalize CPT
```

```

def conditionNodeOnParents(probTensor, node, order):

    axis = order.index(node)

    probTensor /= np.expand_dims(probTensor.sum(axis=axis), axis)

    return probTensor

# Seed
np.random.seed(0)

# Initial CPTs (p)
p = {}

for node in graphNodes:

    dims = [graphNodeNumStates[x] for x in tensorNodeOrder[node]]

    p[node] = randomTensorGenerator(dims)

    p[node] = conditionNodeOnParents(p[node], node, tensorNodeOrder[node])

    print(f"p({node} | {nodeParents[node]}) shape = {p[node].shape}")

# EM initial estimate (phat)
np.random.seed(int(time.time()))

phat = {}

for node in graphNodes:

    phat[node] = randomTensorGenerator(p[node].shape)

    phat[node] = conditionNodeOnParents(phat[node], node, tensorNodeOrder[node])

    print(f"phat({node} | {nodeParents[node]}) shape = {phat[node].shape}")

```

OUTPUT :

p(a | []) shape = (3,)

p(b | []) shape = (4,)

p(c | ['a']) shape = (5, 3)

p(d | ['a','b']) shape = (4, 3, 4)

p(e | ['a','c']) shape = (3, 3, 5)

p(f | ['b','d']) shape = (4, 4, 4)

p(g | ['e']) shape = (5, 3)

p(h | ['f']) shape = (4, 4)

phat(a | []) shape = (3,)

phat(b | []) shape = (4,)

phat(c | ['a']) shape = (5, 3)

phat(d | ['a','b']) shape = (4, 3, 4)

phat(e | ['a','c']) shape = (3, 3, 5)

phat(f | ['b','d']) shape = (4, 4, 4)

phat(g | ['e']) shape = (5, 3)

phat(h | ['f']) shape = (4, 4)

PROGRAM :

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Activation

import numpy as np

# Input and output (XOR problem)
x = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([[0], [1], [1], [0]])

# Create model
model = Sequential()

model.add(Dense(2, input_shape=(2,)))

model.add(Activation('sigmoid'))

model.add(Dense(1))

model.add(Activation('sigmoid'))

# Compile model
model.compile(

    loss='mean_squared_error',

    optimizer='sgd',

    metrics=['accuracy']

)

# Model summary
model.summary()

# Train model (optional but usually expected)
model.fit(x, y, epochs=1000, verbose=0)

# Test prediction
print("Predictions:")

print(model.predict(x))
```


OUTPUT :

./usr/local/lib/python3.11/site-packages/tensorflow/layers/core/utils.py:55: Overriding: Do not pass an "input_shape"/"input_dim" argument to a layer. When using Sequential models, prefer using an "Input(shape)" object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	[None, 20]	6
activation (Activation)	[None, 20]	0
dense_1 (Dense)	[None, 1]	1
activation_1 (Activation)	[None, 1]	0

Total params: 7 (36.00 B)
Trainable params: 7 (36.00 B)
Non-trainable params: 0 (0.00 B)
Predictions:
1/1 -----> 4s 40ms/step
[[0.40375494]
 [0.4057238]
 [0.40540724]
 [0.4073432]]

PROGRAM :

```
from numpy import loadtxt

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# Load dataset (Pima Indians Diabetes)

dataset = loadtxt(

    'https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv',

    delimiter=',

)

# Split into input (X) and output (y)

X = dataset[:, 0:8]

y = dataset[:, 8]

# Create model

model = Sequential()

model.add(Dense(12, input_shape=(8,), activation='relu'))

model.add(Dense(8, activation='relu'))

model.add(Dense(1, activation='sigmoid'))

# Compile model

model.compile(

    loss='binary_crossentropy',

    optimizer='adam',

    metrics=['accuracy'])

# Train model

model.fit(X, y, epochs=150, batch_size=10, verbose=0)

# Make predictions

predictions = (model.predict(X)>0.5).astype(int)

# Display first 5 predictions

for i in range(5):

    print('%s => %d (expected %d))' % (X[i].tolist(), predictions[i][0], y[i]))
```

OUTPUT :

24/24 ————— **0s** 2ms/step

[6.0, 148.0, 72.0, 35.0, 0.0, 33.6, 0.627, 50.0] =>1 (expected 1)

[1.0, 85.0, 66.0, 29.0, 0.0, 26.6, 0.351, 31.0] =>0 (expected 0)

[8.0, 183.0, 64.0, 0.0, 0.0, 23.3, 0.672, 32.0] =>1 (expected 1)

[1.0, 89.0, 66.0, 23.0, 94.0, 28.1, 0.167, 21.0] =>0 (expected 0)

[0.0, 137.0, 40.0, 35.0, 168.0, 43.1, 2.288, 33.0] =>1 (expected 1)